

```

1 #include <Wire.h>
2 #include <Adafruit_ADS1X15.h>
3 #include <ESP32Servo.h>
4 #include <Adafruit_GFX.h>
5 #include <Adafruit_SSD1306.h>
6
7 // ----- OLED -----
8 #define SCREEN_WIDTH 128
9 #define SCREEN_HEIGHT 64
10 #define OLED_ADDR 0x3C
11 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
12
13 // ----- ADC -----
14 Adafruit_ADS1115 ads;
15
16 // ----- SERVOS -----
17 Servo baseServo;
18 Servo topServo;
19 Servo bottomProbe;
20
21 // ----- STEPPER -----
22 #define PULSE_PIN 26
23 #define DIR_PIN 27
24 #define HALL_PIN 33
25
26 const int moveDelay = 1200; // microseconds
27 int currentAngle = 0; // current stepper angle
28
29 // Steps per degree
30 const int stepsPerRev = 200*16; // 16x microstepping
31 const float stepsPerDegree = stepsPerRev / 360.0;
32
33 // ----- BOWLS -----
34 const int bowlAnglesRetrieving[] = {0, 90, 180, 270}; // angles for
    retrieving
35 const int bowlOffsetsSorting[] = {0, 90, 180, 270}; // offsets from
    sorting home
36 const char* resistors[] = {"3.3K", "10K", "100K", "120K"};
37 int indexR = 0;
38
39 // ----- BUTTONS -----
40 #define MODE_BTN 13
41 #define LEFT_BTN 14
42 #define RIGHT_BTN 12
43 #define SELECT_BTN 32
44
45 // ----- SERVOS -----
46 #define BASE_SERVO_PIN 23
47 #define TOP_SERVO_PIN 4
48 #define BOTTOM_SERVO_PIN 5
49
50 // ----- INDICATORS -----
51 #define BUZZER_PIN 17
52 #define BOWL_LED 18
53
54 // ----- BUTTON CONTROL -----
55 unsigned long lastButtonTime = 0;
56 const int buttonDelay = 250;

```

```

57
58 // ----- STATES -----
59 enum State { MODE_SELECT, RETRIEVING, SORTING };
60 State state = MODE_SELECT;
61
62 // ----- BRIDGE CONSTANTS -----
63 float Vin = 3.3;
64 float Rref = 47000.0;
65
66 // ----- RETRIEVING LED -----
67 unsigned long ledStartTime = 0;
68 bool ledActive = false;
69
70 // ----- STARTUP ANIMATION -----
71 #define GEAR_FRAMES 4
72 const unsigned char* gearBitmaps[GEAR_FRAMES] = {
73 /* Add your 4 gear bitmap frames here */
74 };
75
76 // ----- HOMING ANIMATION -----
77 const unsigned char* locBitmaps[2] = {
78 /* Add 2 small location icon frames here */
79 };
80
81 // ----- FUNCTION DECLARATIONS -----
82 void homeMotor();
83 void moveToAngle(int targetAngle);
84 void showMode();
85 void handleModeButton();
86 void showRetrieveUI();
87 void handleRetrieve();
88 void transitionRetrieve(int direction);
89 void handleSorting();
90 void performSorting();
91 void showAnimation(const unsigned char* frames[], int frameCount, int
    duration);
92 void showMessage(String msg);
93 void showMeasuredValue(float Rx);
94
95 // ----- SETUP -----
96 void setup() {
97   Serial.begin(115200);
98   Wire.begin(21,22);
99
100  pinMode(PULSE_PIN, OUTPUT);
101  pinMode(DIR_PIN, OUTPUT);
102  pinMode(HALL_PIN, INPUT_PULLUP);
103
104  pinMode(MODE_BTN, INPUT_PULLUP);
105  pinMode(LEFT_BTN, INPUT_PULLUP);
106  pinMode(RIGHT_BTN, INPUT_PULLUP);
107  pinMode(SELECT_BTN, INPUT_PULLUP);
108
109  pinMode(BUZZER_PIN, OUTPUT);
110  pinMode(BOWL_LED, OUTPUT);
111  digitalWrite(BUZZER_PIN, LOW);
112  digitalWrite(BOWL_LED, LOW);
113

```

```

114 // Servos
115 baseServo.attach(BASE_SERVO_PIN);
116 topServo.attach(TOP_SERVO_PIN);
117 bottomProbe.attach(BOTTOM_SERVO_PIN);
118 baseServo.write(15);
119 topServo.write(90);
120 bottomProbe.write(0);
121
122 // OLED
123 display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
124 display.setTextSize(1);
125 display.setTextColor(SSD1306_WHITE);
126
127 // ADC
128 ads.begin();
129 ads.setGain(GAIN_ONE);
130
131 // ---- Startup animation 2s ----
132 digitalWrite(BUZZER_PIN,HIGH);
133 showAnimation(gearBitmaps, GEAR_FRAMES, 2000); // 2s
134 digitalWrite(BUZZER_PIN,LOW);
135
136 // ---- Credit text 3s ----
137 display.clearDisplay();
138 display.setCursor(0,20);
139 display.setTextSize(1); // normal size to avoid clipping
140 display.println("A product by");
141 display.println("Jubit, Joseph and Noufal");
142 display.display();
143 delay(3000);
144
145 // ---- Homing with location animation ----
146 showMessage("HOMING...");
147 homeMotor(); // run stepper home
148 // Optional: call showAnimation(locBitmaps,2,1000) concurrently during
    homing
149
150 moveToAngle(0); // retrieving home
151 currentAngle = 0;
152
153 showMode();
154 }
155 // ----- LOOP -----
156 void loop() {
157     handleModeButton();
158
159     if(state==RETRIEVING) handleRetrieve();
160     if(state==SORTING) handleSorting();
161
162     // Non-blocking LED for retrieving
163     if(ledActive && millis() - ledStartTime >= 5000){
164         digitalWrite(BOWL_LED,LOW);
165         ledActive = false;
166         showRetrieveUI(); // back to menu
167     }
168 }
169
170 // ----- HOMING -----

```

```

171 void homeMotor() {
172     digitalWrite(DIR_PIN,HIGH);
173     while(digitalRead(HALL_PIN)==HIGH){
174         digitalWrite(PULSE_PIN,HIGH);
175         delayMicroseconds(5000);
176         digitalWrite(PULSE_PIN,LOW);
177         delayMicroseconds(5000);
178     }
179     currentAngle = 0;
180 }
181
182 // ----- MOVE -----
183 void moveToAngle(int targetAngle){
184     int diff = targetAngle - currentAngle;
185     if(diff > 180) diff -= 360;
186     if(diff < -180) diff += 360;
187
188     if(diff >=0) digitalWrite(DIR_PIN,HIGH);
189     else digitalWrite(DIR_PIN,LOW);
190
191     int steps = abs(diff) * stepsPerDegree;
192     for(int i=0;i<steps;i++){
193         digitalWrite(PULSE_PIN,HIGH);
194         delayMicroseconds(moveDelay);
195         digitalWrite(PULSE_PIN,LOW);
196         delayMicroseconds(moveDelay);
197     }
198     currentAngle = targetAngle;
199 }
200
201 // ----- MODE -----
202 void showMode(){
203     display.clearDisplay();
204     display.setCursor(0,10);
205     display.setTextSize(2); // larger for better visibility
206     display.println("Mode Selection");
207     display.setTextSize(1);
208     display.println("-----");
209     display.println("MODE short : Retrieve");
210     display.println("MODE hold  : Sorting");
211     display.display();
212 }
213
214 void handleModeButton(){
215     if(millis()-lastButtonTime < buttonDelay) return;
216     if(digitalRead(MODE_BTN)==LOW){
217         unsigned long t = millis();
218         while(digitalRead(MODE_BTN)==LOW);
219
220         // Beep buzzer on mode change
221         digitalWrite(BUZZER_PIN,HIGH);
222         delay(200);
223         digitalWrite(BUZZER_PIN,LOW);
224
225         if(millis()-t > 1000){
226             // Switch to SORTING
227             state = SORTING;
228             showMessage("SORTING"); // only "Sorting"

```

```

229     moveToAngle(187); // sorting home
230     currentAngle = 187;
231 }else{
232     // Switch to RETRIEVING
233     state = RETRIEVING;
234     moveToAngle(0); // retrieving home
235     currentAngle = 0;
236     showRetrieveUI();
237 }
238 lastButtonTime = millis();
239 }
240 }
241
242 // ----- RETRIEVE -----
243 void showRetrieveUI(){
244     display.clearDisplay();
245     int prev = (indexR-1+4)%4;
246     int next = (indexR+1)%4;
247
248     // Left small text
249     display.setTextSize(1);
250     display.setCursor(0,25);
251     display.println(resistors[prev]);
252
253     // Right small text
254     int textWidth = 6*strlen(resistors[next]);
255     display.setCursor(SCREEN_WIDTH - textWidth,25);
256     display.println(resistors[next]);
257
258     // Center bold
259     display.setTextSize(2);
260     int mainWidth = 12*strlen(resistors[indexR]);
261     display.setCursor(SCREEN_WIDTH/2 - mainWidth/2,25);
262     display.println(resistors[indexR]);
263
264     display.display();
265 }
266
267 void transitionRetrieve(int direction){
268     // Optional smooth sliding animation; implement if desired
269 }
270
271 void handleRetrieve(){
272     if(millis()-lastButtonTime<buttonDelay) return;
273
274     if(digitalRead(LEFT_BTN)==LOW){
275         indexR = (indexR-1+4)%4;
276         showRetrieveUI();
277         lastButtonTime=millis();
278     }
279     if(digitalRead(RIGHT_BTN)==LOW){
280         indexR = (indexR+1)%4;
281         showRetrieveUI();
282         lastButtonTime=millis();
283     }
284
285     if(digitalRead(SELECT_BTN)==LOW && !ledActive){
286         int targetAngle = bowlAnglesRetrieving[indexR];

```

```

287     moveToAngle(targetAngle); // shortest path
288
289     digitalWrite(BOWL_LED,HIGH);
290     ledStartTime = millis();
291     ledActive = true;
292
293     display.clearDisplay();
294     display.setCursor(10,25);
295     display.setTextSize(1);
296     display.println("Take resistor!");
297     display.display();
298
299     lastButtonTime=millis();
300 }
301 }
302
303 // ----- SORTING -----
304 void handleSorting(){
305     if(millis()-lastButtonTime<buttonDelay) return;
306     if(digitalRead(SELECT_BTN)==LOW){
307         performSorting();
308         lastButtonTime=millis();
309     }
310 }
311
312 void performSorting(){
313     topServo.write(0);
314     bottomProbe.write(90);
315     delay(2000);
316
317     long total=0;
318     for(int i=1;i<49;i++){
319         total += ads.readADC_SingleEnded(1);
320         delay(5);
321     }
322     float adc = total/48.0;
323     float V1 = adc * 0.000125;
324     float Rx = (V1*Rref)/(Vin-V1);
325
326     topServo.write(90);
327     bottomProbe.write(0);
328     delay(1000);
329     // Show measured value in large text
330     showMeasuredValue(Rx);
331
332     int bowlIndex;
333     if(Rx < 5500) bowlIndex = 0;
334     else if(Rx < 50000) bowlIndex = 1;
335     else if(Rx < 105000) bowlIndex = 2;
336     else bowlIndex = 3;
337
338     int targetAngle = (187 + bowlOffsetsSorting[bowlIndex]) % 360;
339     moveToAngle(targetAngle);
340
341     baseServo.write(90);
342     delay(2000);
343     baseServo.write(15);
344 }

```

```

345
346 void showMeasuredValue(float Rx){
347     display.clearDisplay();
348     display.setTextSize(2);
349     display.setCursor(20,20);
350     display.print(Rx,0); // show integer value
351     display.println(" Ohm");
352     display.display();
353     delay(2000);
354
355     // Show "Next resistor" in small text
356     display.clearDisplay();
357     display.setTextSize(1);
358     display.setCursor(10,25);
359     display.println("Next resistor");
360     display.display();
361     delay(1500);
362 }
363
364 // ----- ANIMATION -----
365 void showAnimation(const unsigned char* frames[], int frameCount, int
    duration){
366     int frameTime = duration / frameCount;
367     for(int i=0;i<frameCount;i++){
368         display.clearDisplay();
369         // display.drawBitmap(0,0,frames[i], SCREEN_WIDTH, SCREEN_HEIGHT,
            SSD1306_WHITE);
370         display.display();
371         delay(frameTime);
372     }
373 }
374
375 // ----- MESSAGE -----
376 void showMessage(String msg){
377     display.clearDisplay();
378     display.setTextSize(2); // larger messages
379     display.setCursor(10,25);
380     display.println(msg);
381     display.display();
382 }

```